PRODUCT REQUIREMENTS DOCUMENT • VERSION 1.0

Mini Marketplace

A multi-vendor commerce platform

Build and deliver a working marketplace where customers purchase products from multiple sellers, sellers manage inventory and fulfillment, and administrators oversee marketplace operations.

Delivery horizon	Core capabilities
4 weeks	DDD · Docker · AWS · Messaging · CI/CD

Purpose

Deliver a maintainable application that can be run locally, deployed to AWS, monitored, tested, and safely extended by another developer. The emphasis is on a complete end-to-end delivery—not visual polish or the number of features implemented.

Working approach

Treat this as a product delivery. Clarify business rules before committing to an implementation, keep a prioritised backlog, demonstrate progress regularly, and raise scope or delivery risks early. Technical decisions should be documented and justified.

1. Business overview

Mini Marketplace connects three primary user groups. Customers browse and purchase products; sellers manage their catalogue, stock and fulfillment; administrators manage marketplace operations.

- **Customers** browse products, manage a cart, place orders, complete a simulated payment, and track order status.
- **Sellers** manage their own products, prices, inventory and seller-specific order fulfillment.
- **Administrators** manage users, seller approvals, categories, orders and operational issues.

A customer must be able to purchase products from multiple sellers in one checkout. The platform must handle inventory correctly, track order and payment outcomes, and perform background work reliably.

2. User roles and permissions

Role	Responsibilities
Guest	Browse and search the product catalogue.
Customer	Manage profile and addresses, cart, checkout, payments and orders.
Seller	Manage own products, stock and seller-specific orders.
Administrator	Manage sellers, categories, users and marketplace operations.

Security requirement: Permissions must be enforced on the server. Hiding a UI control is not authorization. Sensitive actions must be attributable to the user or system process that performed them.

3. Functional requirements

FR-01 — Authentication and authorization

- Users can register, log in and log out.
- Customers can manage their profiles and addresses.
- Sellers can access seller functionality only after approval.
- Administrators can manage user and seller accounts.
- Unauthorized actions are rejected; sensitive actions record the actor.

FR-02 — Product catalogue

- Sellers can create, update and deactivate their own products.
- Each product has a name, description, SKU, category, price, images and stock quantity.
- Products belong to a seller and a category.
- Customers can browse, search, filter, sort and view product details and availability.
- Inactive or unavailable products cannot be purchased. Price and stock are revalidated at checkout.

FR-03 — Shopping cart

- Customers can add products, remove products and change quantities.
- The cart displays item prices, quantities, subtotals and the total.
- The system handles products becoming unavailable or changing price after being added to the cart.
- Cart contents are associated with the correct customer. Checkout recalculates prices and validates stock; it does not trust client-supplied totals.

FR-04 — Checkout and orders

- Customers can check out a cart containing products from multiple sellers.
- Create one overall checkout/order record and represent seller-specific fulfillment separately as appropriate.
- Record purchased items, quantities, seller, unit prices, applicable discounts and totals.

- Historical order prices remain unchanged if a seller later changes a product price.
- Repeated checkout submissions must not create duplicate orders.
- Customers can view their order history and details, but cannot access another customer's orders.

FR-05 — Inventory management

- Sellers can update their own inventory.
- Inventory is validated and protected against overselling, including concurrent purchases.
- Define when stock is reserved, committed and released.
- Failed, cancelled or expired orders must not leave stock permanently reserved.
- Inventory changes must be traceable.

Required design note: Document the inventory reservation rules and explain how they behave when requests overlap or a workflow fails.

FR-06 — Payments

- Use a payment sandbox or a simulated payment provider; real payment processing is not required.
- Support success, failure, pending and unknown outcomes.
- Store payment references and status; do not store raw payment card details.
- Repeated payment requests must not accidentally charge a customer twice.
- Associate payment status with the correct order and define recovery for ambiguous outcomes.

FR-07 — Order fulfillment

- Sellers can view orders containing their products.
- Sellers can update fulfillment status only through valid state transitions.
- Customers can track order status.
- Invalid transitions are rejected.
- Order cancellation triggers appropriate inventory and payment handling.
- A multi-seller checkout tracks each seller's fulfillment independently.

FR-08 — Administration

- Administrators can approve or suspend sellers and manage product categories.
- Administrators can inspect orders and payment statuses.
- Administrators can investigate failed background jobs and operational issues.
- Administrative actions are logged.

FR-09 — Notifications and background processing

- Publish an order-related event after an order is successfully created.
- Send an order confirmation through a mock email service or another testable notification mechanism.
- Process notifications asynchronously so delivery does not block checkout.
- Retry transient failures and route repeatedly failing messages to a dead-letter queue (DLQ).
- Prevent duplicate message processing from causing duplicate business effects.
- Log failures and provide a way to investigate and recover failed messages.

Required design note: Explain how the system reliably publishes an event when a database transaction succeeds but the application fails before publishing the corresponding message.

FR-10 — Audit trail

Record important business actions, including:

- Product price and inventory changes.
- Order and payment status changes.
- Seller approval or suspension.
- Administrative actions.

Each audit record should include the action, relevant entity, timestamp and actor or system process.

4. Non-functional requirements

Area	Expectation
Maintainability	Clear separation of responsibilities and understandable code.
Architecture	Apply DDD where useful; document the domain model and design decisions.
Data integrity	Enforce business invariants and appropriate transaction boundaries.
Security	Enforce authorization, validate inputs, protect secrets and avoid exposing sensitive data.
Reliability	Handle retries, duplicate messages, partial failures and recovery.
Performance	Propose reasonable response-time targets and explain assumptions; no large-scale load is expected.
Observability	Structured logs, health checks, error reporting and queue monitoring.
Testability	Automated tests for business rules, integration boundaries and critical workflows.
Documentation	Another developer can set up, deploy, troubleshoot and maintain the application.

5. Technical and infrastructure requirements

Application architecture

Apply DDD where it adds value, document the main domain concepts and boundaries, and justify the chosen architecture. A modular monolith is a reasonable starting point; microservices are not required.

Database

Use a persistent relational database. Provide versioned schema migrations, suitable constraints and indexes, and a documented approach to transaction management.

Docker

Provide a reproducible local environment using Docker Compose or an equivalent documented setup. Another developer must be able to run the application, database and background worker using the documented steps.

AWS deployment

Deploy the application and required supporting services to AWS. Document infrastructure, configuration, networking, permissions, database access, secrets and cost considerations. Use least-privilege access and avoid committing secrets.

CI/CD

Implement an automated pipeline that runs checks and tests, builds the application, and deploys approved changes. Document deployment verification and rollback procedures.

Messaging

Use a queue or broker suitable for AWS deployment. Document message contracts, retry behavior, idempotency, dead-letter handling and worker lifecycle.

6. Testing requirements

Automated tests must cover at least the following scenarios:

- Customers cannot access another customer's orders.

- Sellers cannot modify another seller's products.
- Product price changes do not alter existing orders.
- Concurrent purchases cannot oversell inventory.
- Repeated checkout submissions do not create duplicate orders.
- Repeated message delivery does not cause duplicate business effects.
- Failed notifications are retried and eventually routed to a DLQ.
- Payment timeouts and retries do not create duplicate charges in the simulated integration.
- Invalid order state transitions are rejected.
- A failed deployment or unhealthy application can be detected.

Unit tests alone are insufficient. Critical workflows require integration tests, and concurrency-sensitive behavior must be tested explicitly.

7. Delivery and maintenance

The project is not complete merely because it works on one machine. Final delivery must include:

- Source code in a version-controlled repository.
- A documented branching and pull-request workflow.
- An automated build and test pipeline.
- Database migrations and sample seed data.
- Docker-based local development setup.
- A deployed AWS environment and deployment instructions.
- An architecture diagram and explanation of major decisions.
- API documentation, if an API is exposed.
- Known limitations and outstanding work.
- Logs, health checks and basic operational dashboards or equivalent monitoring.
- A backup and recovery approach.
- Deployment, rollback and troubleshooting runbooks.
- A demonstration of at least one failure, its diagnosis and its resolution.

8. Four-week delivery plan

Week	Focus	Expected outcome
1	**Discovery and design**	Clarified requirements, user stories, acceptance criteria, domain model, architecture decisions, database design, initial backlog and deployment plan.
2	**Working MVP**	Authentication, catalogue, seller permissions, cart, checkout, order management, inventory rules, critical tests and initial asynchronous notification workflow.
3	**Production readiness**	AWS deployment, CI/CD, queue reliability, retries, DLQ handling, secrets, observability, migrations, backup planning and documentation.
4	**Maintenance and change**	New requirements and introduced defects; root-cause analysis, code changes, regression tests, deployment, verification and documentation.

This is a suggested sequence, not a requirement to implement every feature immediately. Prioritise a complete, reliable end-to-end checkout flow over visual polish. Raise delivery risks early and propose scope reductions with clear reasoning.

9. Acceptance criteria

The project is considered successfully delivered when:

- A customer can purchase products from multiple sellers in one checkout.
- Sellers can manage their own products and fulfill their own portions of orders.
- Inventory and order state remain consistent under concurrent requests.
- Payment outcomes are handled safely in the simulated integration.
- Background jobs support retries, duplicate-delivery protection and dead-letter handling.
- Critical workflows have automated tests.
- The application and workers run locally using the documented setup.
- The application is deployed to AWS through an automated pipeline.
- Logs, health checks and operational troubleshooting procedures are available.
- Another developer can understand the architecture and maintain the system.

- An introduced defect can be diagnosed and fixed without relying on a rewrite.

10. Product delivery and working expectations

- Actively clarify ambiguous business requirements before implementation.
- Maintain a prioritised backlog with small, understandable tickets and acceptance criteria.
- Use pull requests for changes and respond constructively to review feedback.
- Demonstrate working software regularly rather than waiting until the end.
- Keep technical decisions and known limitations documented.
- Identify risks early and explain trade-offs when proposing a change in scope.

Changes and operational scenarios

During the final week, expect additional scenarios such as:

- **Business change:** Sellers can cancel their own portion of a multi-seller order without cancelling the entire checkout.
- **Production defect:** The final unit of a SKU was assigned to two customers. Investigate the evidence, identify the root cause, fix it and add a regression test.
- **Operational incident:** A poison message repeatedly fails in the worker. Explain why it failed, how to prevent endless retries, and how to recover safely.

For each scenario, demonstrate investigation, a reasoned fix, automated verification, deployment and post-deployment checks. Document relevant findings and any follow-up work.

11. Design decisions to own

The requirements intentionally define business outcomes rather than prescribing the implementation. You are responsible for proposing and explaining:

- Bounded contexts, aggregates, domain events and the domain model.
- Database schema, transaction boundaries and concurrency strategy.
- Message contracts, event-delivery strategy and messaging topology.
- Inventory reservation and release rules.
- Payment workflow, idempotency and ambiguous-outcome recovery.
- AWS architecture and infrastructure-as-code approach.
- CI/CD stages, deployment verification and rollback strategy.
- Monitoring, incident response and recovery procedures.

> **Definition of success**
> A successful delivery is not the one with the most features. It is a working, secure and maintainable end-to-end flow, supported by tests and operational documentation, that can be deployed, diagnosed and safely changed by another developer.